1. Problem Definition and Task Understanding

The goal of this case study is to build a sequence-based menstrual cycle forecasting model for two related daily-level tasks: classifying the current menstrual phase and estimating the number of days remaining until the next cycle begins.

The problem is formulated as a multi-task sequence learning task. The classification head predicts one of four phase labels: menstruation, follicular, ovulation, and luteal. The regression head predicts days_until_next_cycle, which represents the remaining time until the next recorded cycle start date. Since both targets depend on the user's position within the menstrual cycle, a shared temporal encoder is used to learn a common representation of cycle progression, followed by two task-specific output heads.

A key challenge is data transformation. The raw dataset is cycle-level, where each row represents one menstrual cycle, while the model requires daily time-window sequences. Therefore, the data must be expanded into daily records, enriched with temporal and historical features, and converted into sliding-window sequences. During this process, special care is required to avoid target leakage, especially because variables such as current cycle length and period length are directly related to target and label construction.

Overall, this case is not only a modeling task, but also a reasoning task about sequence construction, leakage prevention, feature encoding, and evaluation under uncertainty.

2. Dataset Overview and EDA Findings

The dataset is a synthetic menstrual-cycle tracking dataset containing repeated cycle records for 100 users. Each raw row represents one recorded menstrual cycle and includes demographic variables, lifestyle-related factors, temporal cycle information, and symptoms: User ID, Age, BMI, Stress Level, Exercise Frequency, Sleep Hours, Diet, Cycle Start Date, Cycle Length, Period Length, Next Cycle Start Date, and Symptoms.

Although the assignment is sequence-oriented, the raw file is not directly organized as daily time-series data. Therefore, the first step was to inspect the dataset and design a daily expansion and sequence construction pipeline. The detailed exploratory analysis is included in the project repository as an EDA notebook, while the main findings are summarized here.

The initial EDA showed that the dataset is clean: there are no missing values, date columns are parseable, and cycle records can be chronologically ordered within each user. No users had too few cycles to be used in the sequence modeling pipeline, which made it possible to apply a user-level train/validation/test split without removing low-history users.

A key finding was that Cycle Length is consistent with the difference between Cycle Start Date and Next Cycle Start Date. This confirmed that days_until_next_cycle can be derived reliably after daily expansion. However, it also revealed a leakage risk: using the current Cycle Length as a model input would make the regression task artificially easy, and Period Length directly defines the menstruation boundary in the rule-based phase labeling process. Therefore, both variables were kept for audit, target construction, and label generation, but excluded from the final model input features.

EDA also showed that several variables behave mostly as user-level or cycle-level context rather than daily-changing signals. Age, BMI, exercise frequency, diet, stress level, and sleep hours are repeated across cycle records in this synthetic dataset. These variables were still retained because the assignment requires all fields to be used, and because they can provide contextual information about cycle rhythm and symptoms.

Overall, the EDA led to three main preprocessing decisions: expand the raw cycle-level data into daily records, exclude leakage-prone current-cycle variables from model inputs, and split the data at the user level rather than the row level.

3. Data Preparation and Leakage Prevention

The data preparation pipeline was designed to convert the raw cycle-level dataset into a leakage-safe daily sequence dataset. It consists of three main steps: daily dataset construction, user-level preprocessing, and sliding-window sequence generation.

3.1 Daily Dataset Construction

First, raw column names were normalized and date columns were parsed. Cycle records were then sorted chronologically within each user, and duplicate (user_id, cycle_start_date) pairs were checked. This ordering step is important because historical aggregation features must only use cycles that occurred before the current cycle.

Each cycle-level row was then expanded into daily rows. For every cycle, day_in_cycle was generated from 0 to cycle_length - 1, and sample_date was computed by adding this offset to the cycle start date. The regression target, days_until_next_cycle, was calculated as the difference between Next Cycle Start Date and the daily sample_date.

3.2 Historical Features and Cold-Start Handling

To add user-specific context without leaking information from the current cycle, I created two historical features: hist_mean_cycle and hist_mean_period. These were calculated using only previous cycles of the same user through a shifted expanding mean. This allows the model to access a user's past cycle rhythm while preventing the current cycle length or period length from leaking into the input.

For the first cycle of each user, where no personal history exists, I used population-level cold-start priors from Bull et al. (2019), a large-scale mobile health study summarizing more than 600,000 menstrual cycles: 29.3 days for mean cycle length and 4.0 days for mean period length. I also added a binary flag, is_historical_data_missing, to indicate when these prior values were used instead of user-specific history. This flag helps the model distinguish between true user-specific historical averages and imputed cold-start values, rather than treating both as equally reliable personal history.

3.3 Phase Label Generation

Phase labels were generated using a rule-based approximation. Days before Period Length were labeled as menstruation. Ovulation was approximated as occurring 14 days before the next cycle onset, using ovulation_day = cycle_length - 14. This choice is based on the common observation that the luteal phase is relatively more stable across cycles, while much of the variation in total cycle length is driven by variation in the follicular phase. Therefore, estimating ovulation relative to the next cycle start was more appropriate than assigning it to a fixed calendar day such as day 14. Days between menstruation and ovulation were labeled as follicular, and days after ovulation were labeled as luteal.

Before applying this rule, the cycle-length and period-length ranges were checked to ensure that the rule did not create invalid phase boundaries. Since the dataset does not include hormonal measurements or ovulation-test data, these phase labels should be interpreted as approximate derived labels rather than perfect biological ground truth.

3.4 Categorical Encoding

Exercise Frequency was encoded ordinally because it has a natural order: Low, Moderate, and High. Diet and Symptoms were represented as binary one-hot input features to avoid imposing artificial ordinal relationships between categories. During EDA, both diet and symptom fields behaved as single-label categorical variables: each record contained one diet category and one symptom category rather than multiple simultaneous labels. Therefore, one-hot encoding was sufficient. In a real tracking application, users may report multiple symptoms on the same day, in which case multi-hot encoding would be more appropriate.

I preferred ordinal and one-hot encoding over embedding layers because the categorical variables have very low cardinality. In this small synthetic dataset, trainable embeddings would add extra parameters without a clear benefit and could increase overfitting risk. The resulting 0/1 indicators were kept unscaled, allowing each category to remain explicitly visible to the LSTM at every timestep.

3.5 Leakage Prevention

A central leakage-prevention decision was to exclude current cycle_length and period_length from the final model input features. Although both fields were necessary for target and label construction, they would leak information if used directly as predictors. Current cycle_length almost directly determines days_until_next_cycle, while period_length directly defines the menstruation boundary. Therefore, these columns were retained in the processed dataset for auditability but removed from the model input feature list.

3.6 User-Level Split and Scaling

After daily expansion and feature encoding, the dataset was split at the user level into train, validation, and test sets using a 70/15/15 ratio. This prevents cycles from the same user appearing in both training and evaluation splits, which would otherwise allow the model to learn user-specific cycle patterns and produce overly optimistic evaluation results.

Continuous input features were normalized using MinMax scaling. The scaler was fitted only on the training split and then applied to validation and test splits. Targets were not scaled: days_until_next_cycle was kept in days so that regression MAE remained directly interpretable in real-world terms. Binary and one-hot encoded categorical columns were also left unscaled.

4. Sequence Construction

After the daily-level dataset was created and preprocessed, it was converted into fixed-length time-window sequences for the LSTM model. The sequence construction step transforms daily rows into three-dimensional input arrays with the shape (n_windows, sequence_length, n_features), where each sample represents a consecutive window of daily observations.

4.1 Sliding-Window Strategy

I used a sliding-window strategy rather than non-overlapping chunks. For a given cycle, a window of length seq_length is moved one day at a time. For example, if seq_length = 14, a 28-day cycle produces windows covering days 0–13, 1–14, 2–15, and so on. This increases the number of training samples while preserving the local temporal order of cycle progression.

Windows were generated within each (user_id, cycle_start_date) group. This means that a sequence never crosses from one user to another or from one cycle to the next. This design avoids mixing unrelated temporal contexts and prevents the model from learning artificial transitions between separate cycles. Within each group, rows were sorted by day_in_cycle before window creation.

Static and cycle-level variables such as age, BMI, diet, exercise frequency, stress level, sleep hours, symptoms, and historical averages were repeated across all daily timesteps belonging to the corresponding cycle. This allows the LSTM to receive the same contextual information at each timestep, while temporal progression is represented by day_in_cycle and the ordered daily sequence itself.

4.2 Sequence-to-Sequence Target Format

Both tasks were formulated in a sequence-to-sequence format. Each timestep inside a window has its own phase label and its own days_until_next_cycle value. Therefore, the model does not produce only one prediction per window; it predicts a phase and a remaining-day value for every day in the input sequence.

The resulting arrays follow this structure:

- X: (n_windows, seq_length, n_features)
- y_phase: (n_windows, seq_length)
- y_reg: (n_windows, seq_length, 1)

This format matches the multi-output LSTM design: the shared encoder processes the daily sequence, and the two output heads generate timestep-level predictions for classification and regression.

4.3 Sequence Length Selection

I tested two sequence lengths: 14 days and 21 days. A 14-day window was a natural baseline because menstrual-cycle reasoning often uses a two-week structure, especially around the approximation that ovulation occurs about 14 days before the next cycle onset. However, a 14-day window may not contain enough context to capture broader phase transitions or cycle rhythm.

A 21-day window provides a longer temporal context while still remaining shorter than most complete cycles in the dataset. This allows the model to observe a larger portion of the current cycle, including more gradual transitions between phases. Since menstrual-cycle progression is a slowly changing temporal process, longer context may help the LSTM learn smoother phase progression and improve next-cycle countdown estimation.

Both sequence lengths were evaluated experimentally, and the final choice is discussed in the model selection section.

5. Multi-Task LSTM Architecture

The model was designed as a shared-input multi-task LSTM with two output heads. This follows the assignment requirement that both tasks should be solved by one model with a shared encoder rather than by two independent models.

5.1 Shared Temporal Encoder

The input to the model is a daily sequence with shape (seq_length, n_features). Each timestep represents one day, and each feature vector contains temporal, demographic, lifestyle, symptom, and historical context features.

I used an LSTM layer as the shared temporal encoder because the task depends on ordered cycle progression. The LSTM receives the daily window and learns a representation of how the user moves through the menstrual cycle over time. This is useful because both target tasks depend on the same underlying temporal structure: the current phase and the remaining time until the next cycle are both related to the user's position within the cycle.

The LSTM was configured with return_sequences=True, because the model needs to produce predictions for every timestep in the input window, not just one prediction at the end of the sequence.

5.2 Multi-Task Output Heads

After the shared LSTM encoder, I used a small dense layer to transform the shared temporal representation before branching into two task-specific heads.

The first head is the phase classification head. It uses a softmax output with four classes: menstruation, follicular, ovulation, and luteal. Since this prediction is required at every timestep, the dense classifier is applied in a time-distributed manner.

The second head is the next-cycle onset regression head. It outputs one continuous value per timestep, representing days_until_next_cycle. This head uses a linear output activation because the target is a numeric value measured in days.

This architecture allows the model to learn a shared representation of cycle progression while still optimizing separate output formats for the two tasks: categorical phase labels and continuous remaining-day estimates.

5.3 Why Multi-Task Learning

Two separate LSTM models could technically be trained: one for phase classification and one for next-cycle regression. However, this would require both models to learn similar temporal cycle dynamics independently. Since the two tasks are closely related, I used hard parameter sharing: the LSTM encoder is shared, while the output heads remain task-specific.

This setup can improve efficiency and encourage the model to learn more general cycle representations. For example, learning where a user is within the cycle can help both phase classification and countdown estimation. At the same time, keeping separate heads allows each task to use its own loss function and output type.

5.4 Loss Functions and Loss Weighting

The classification head was trained using sparse categorical cross-entropy, because the phase labels are integer class labels rather than one-hot target vectors. The regression head was trained using mean absolute error, because the target is measured in days and MAE is directly interpretable.

The total training loss was defined as a weighted sum of the two task losses:

*total_loss = phase_loss + 0.1 * regression_loss*

This weighting was necessary because the regression MAE values are numerically larger than the classification loss. Without weighting, the regression task could dominate the shared encoder updates and reduce the model's ability to learn phase classification. A smaller regression weight keeps both tasks active during training while preserving the interpretability of the regression metric in days.

5.5 Regularization

Dropout was added after the LSTM layer to reduce overfitting. The early baseline models showed that validation performance stopped improving after a few epochs while training performance continued to increase, indicating overfitting. A dropout rate of 0.2 improved the balance between training and validation behavior and was retained in the final model.

6. Experiments, Evaluation, and Model Selection

This section summarizes the experimental process used to select the final model. Rather than performing uncontrolled hyperparameter tuning, I followed a controlled comparison strategy: starting from a baseline multi-task LSTM, I changed one main factor at a time and evaluated how each choice affected both phase classification and next-cycle regression.

6.1 Evaluation Metrics

The model was evaluated on both classification and regression outputs. For phase classification, I used accuracy, precision, recall, F1-score, macro F1, weighted F1, and confusion matrices. Accuracy and weighted F1 are useful for measuring overall performance, but they can be dominated by majority classes. Therefore, macro F1 was also included to better inspect minority-class behavior, especially because the ovulation phase is represented as a one-day class and is much less frequent than follicular or luteal phases.

For next-cycle onset regression, I used MAE and RMSE. MAE was the primary regression metric because it is directly interpretable in days. For example, an MAE of 5 days means that the model's next-cycle countdown prediction is, on average, approximately 5 days away from the target value. RMSE was also tracked because it is more sensitive to larger errors.

The regression target was not scaled during preprocessing, so regression metrics could be interpreted directly in day units.

6.2 Experimental Strategy

The experiments were designed to answer the optimization requirements of the assignment while keeping the comparison interpretable. All models used the same user-level train/validation/test split and the same multi-task evaluation protocol.

The experiments were organized as controlled pairwise comparisons. The main idea was to keep all other settings fixed and change only one design factor at a time, so that the effect of that specific factor could be interpreted more clearly. After each comparison, the better-performing option was used as the basis for the next experiment.

For example, I first compared 14-day and 21-day sequence windows under the same baseline architecture. Since the 21-day window performed better, it became the base setting for the following experiments. Then I evaluated the effect of dropout, activation function, optimizer, and batch size in the same controlled manner.

One exception was the ReLU experiment. After testing ReLU with Glorot/Xavier initialization, I also tested ReLU with He initialization because He initialization is commonly more suitable for ReLU-based networks. This additional experiment was included to avoid rejecting ReLU based only on an initializer that may not be optimal for it.

The following design choices were compared:

- sequence length: 14 days vs. 21 days
- dropout: no dropout vs. dropout rate 0.2

- dense activation: tanh vs. ReLU
- initializer: Glorot/Xavier vs. He
- optimizer: Adam vs. mini-batch SGD
- batch size: 32 vs. 64

The purpose was not only to find the best numerical result, but also to understand why each modeling choice affected learning.

6.3 Baseline Progression

I first trained a baseline multi-task LSTM using a 14-day sequence length. This model provided an initial reference point for both tasks.

Next, I increased the sequence length from 14 to 21 days while keeping the same baseline architecture. This improved the overall results, suggesting that a longer temporal window was useful for capturing broader menstrual-cycle progression.

After selecting the 21-day setting as the stronger baseline, I added dropout with a rate of 0.2 and used this configuration as the main candidate for the remaining optimization comparisons.

6.4 Optimization Experiment Results

Experiment	Changed factor	Test accuracy	Macro F1	Weighted F1	Test MAE (days)	Interpretation
baseline_seq14	Initial baseline	0.755	0.552	0.738	5.719	Initial reference model
baseline_seq21	Sequence length 14 → 21	0.784	0.556	0.771	5.252	Longer context improved performance
seq21_tanh_glorot_adam_bs32_dropout02	Dropout 0.2	0.792	0.559	0.775	5.138	Best overall balance
seq21_relu_glorot_adam_bs32_dropout02	tanh → ReLU	0.788	0.558	0.768	5.158	Close, but weaker than tanh
seq21_relu_he_adam_bs32_dropout02	ReLU + He initializer	0.776	0.461	0.750	5.242	Worse minority-class behavior
seq21_tanh_glorot_sgd_bs32_dropout02	Adam → SGD	0.690	0.243	0.589	9.238	Much slower and weaker convergence
seq21_tanh_glorot_adam_bs64_dropout02	Batch size 32 → 64	0.789	0.556	0.766	5.277	Did not improve over batch size 32
seq14_tanh_glorot_adam_bs32_dropout02	Final config repeated with seq14	0.749	0.551	0.734	5.640	Sanity check; seq21 still better

Table 1 Optimization Experiment Results on the Test Set

The sequence-length comparison showed that the 21-day window was more effective than the 14-day window. Although shorter windows generate more training samples, the 21-day window provided more useful temporal context for this task. After finding the best-performing configuration with the 21-day setting, I also repeated the same final configuration with a 14-day window as a final sanity check. The reason was that 14-day windows still had a potential advantage: they produced more training samples and could, in principle, reduce unnecessary temporal noise. However, the 14-day version of the final configuration still underperformed the 21-day version, confirming that longer context was more useful than the larger number of shorter windows in this dataset.

Adding dropout improved the balance between training and validation behavior. The improvement was not dramatic, but it was consistent enough to keep dropout rate 0.2 in the final configuration.

The activation and initializer experiments showed that tanh with Glorot/Xavier initialization was more suitable than the ReLU-based alternatives. Although ReLU with Glorot performed reasonably close, it did not outperform the tanh configuration. ReLU with He initialization performed worse, especially in macro F1, suggesting weaker minority-class behavior.

The optimizer experiment showed a clear difference between Adam and SGD. With the same learning rate and batch size, SGD converged much more slowly and produced much weaker classification and regression results. Adam was therefore retained because it provided faster and more stable optimization for the multi-output LSTM.

The batch-size experiment showed that increasing batch size from 32 to 64 did not improve performance. Batch size 32 gave a better overall balance across classification and regression metrics, so it was kept in the final model.

6.5 Final Model Selection

Based on the controlled experiments, the final selected model was:
seq21_tanh_glorot_adam_bs32_dropout02

The final configuration was:

- sequence length: 21 days
- LSTM units: 64
- dense units: 32
- dropout rate: 0.2
- dense activation: tanh
- kernel initializer: Glorot/Xavier
- recurrent initializer: orthogonal
- optimizer: Adam
- batch size: 32
- loss weights: phase output = 1.0, regression output = 0.1

This configuration was selected because it achieved the best overall balance between phase classification and next-cycle regression. It combined the temporal advantage of the 21-day window, the regularization benefit of dropout, and the stable optimization behavior of Adam.

6.6 Final Evaluation Interpretation

The final model achieved a test accuracy of approximately 0.792, weighted F1 of 0.775, macro F1 of 0.559, and regression MAE of approximately 5.14 days.

The weighted F1 score suggests that the model learned the dominant phase patterns reasonably well. However, the lower macro F1 indicates that performance was not balanced across all classes. This difference is important because the ovulation class is much rarer than the other phases and is represented as a one-day label.

For the regression task, an MAE of 5.14 days means that the model's remaining-days prediction is, on average, about five days away from the target. This is useful as a baseline sequence model, but for a real health application, a point estimate alone would not be sufficient. A calibrated prediction interval would be more appropriate for communicating uncertainty around the next expected cycle start.

6.7 Confusion Matrix Interpretation

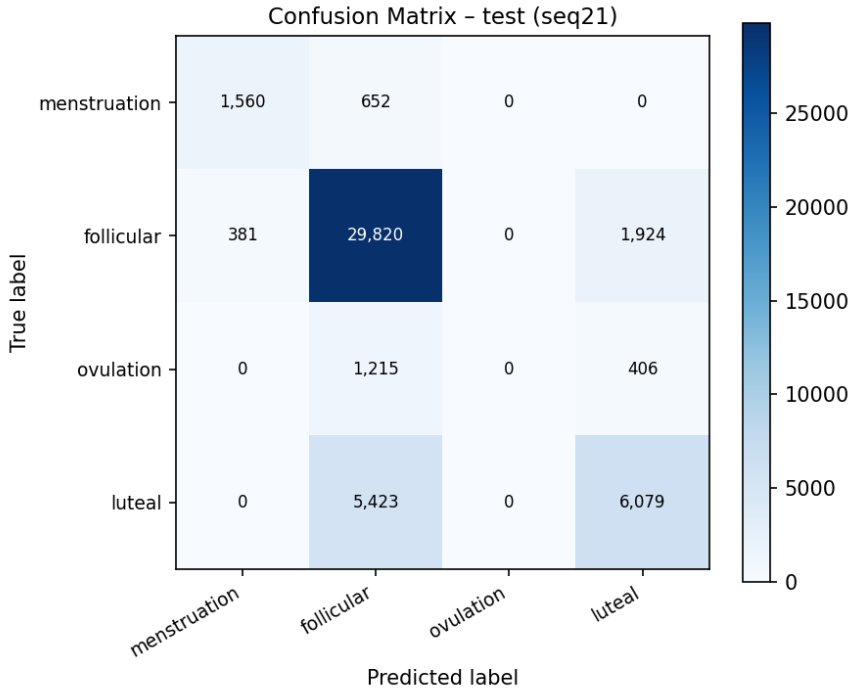

Figure 1. Confusion Matrix of the Final Model on the Test Set

The confusion matrix shows that the model learned menstruation, follicular, and luteal phases to some extent, but it failed to predict the ovulation class. Most ovulation timesteps were assigned to neighboring follicular or luteal phases. This is not surprising because

ovulation is represented as a one-day minority class, while follicular and luteal phases cover much longer intervals.

This explains why weighted F1 remained relatively high while macro F1 stayed lower. Weighted F1 is influenced more by frequent classes, whereas macro F1 penalizes poor performance on minority classes. Therefore, macro F1 is important in this case because it reveals a clinically meaningful weakness that accuracy alone would hide.

The ovulation result should also be interpreted in the context of the label construction strategy. Since the dataset does not include direct biological measurements such as hormone levels, ovulation tests, basal body temperature, or cervical mucus, ovulation labels were rule-based approximations. Future versions of the model could address this limitation using class weighting, focal loss, oversampling around phase transitions, or richer physiological input signals.

7. Sequence-Level Error Analysis

I also inspected individual predictions at the sequence level. Each row in Table 2 represents one selected 21-day test window, not a full menstrual cycle. To keep the table readable, daily phase labels are summarized as phase intervals, and the regression task is reported using sequence-level next-cycle MAE in days.

Example	User	Phase acc.	Next-cycle MAE (days)	True phase intervals	Predicted phase intervals	Context	Ovulation in window	Phase transition
Correct	37	1.00	1.30	Day 0–20: follicular	Day 0–20: follicular	Diet: Balanced; Symptom: Headache; Exercise: Low	0	0
Correct	67	1.00	1.43	Day 0–2: menstruation; Day 3–20: follicular	Day 0–2: menstruation; Day 3–20: follicular	Diet: Balanced; Symptom: Bloating; Exercise: Moderate	0	1
Incorrect	36	0.19	11.50	Day 0–2: menstruation; Day 3–6: follicular; Day 7: ovulation; Day 8–20: luteal	Day 0–20: follicular	Diet: Vegetarian; Symptom: Fatigue; Exercise: High	1	1
Incorrect	78	0.29	13.47	Day 0: menstruation; Day 1–6: follicular; Day 7: ovulation; Day 8–20: luteal	Day 0–20: follicular	Diet: Balanced; Symptom: Headache; Exercise: Low	1	1

Table 2. Examples of Correct and Incorrect Sequence-Level Predictions

The correct examples show that the model can predict stable phase patterns and simple transitions accurately. In contrast, the incorrect examples both include ovulation and phase transitions, but the model predicts the full window as follicular. This matches the confusion matrix findings: ovulation is difficult to detect because it is a one-day minority class and is defined using an approximate rule-based label.

The context column is included only as descriptive background. Since diet, symptoms, and exercise frequency are cycle-level values in this synthetic dataset, they should not be interpreted as direct causes of the errors. Overall, the sequence-level examples support the main evaluation result: the model learns broad phase progression reasonably well, but struggles around short transition regions, especially ovulation.

8. Literature Connection, Limitations, and Future Work

The literature review supports the main design choices and limitations of this project. Prior menstrual-cycle prediction studies show that LSTM-based models are suitable for learning temporal cycle patterns, while multi-task learning literature supports using a shared encoder with task-specific heads when related targets depend on the same underlying sequence representation.

At the same time, real-world menstrual tracking data is more complex than this synthetic dataset. Previous mobile-health studies highlight issues such as skipped logging, cycle variability, symptom variation, and uncertain ovulation timing. In this project, these problems were not explicitly modeled because the dataset is clean and does not contain daily adherence behavior, hormonal measurements, ovulation tests, or wearable signals. Therefore, the generated phase labels should be interpreted as approximate rule-based labels rather than perfect biological ground truth.

Future work could improve the model by adding class weighting or focal loss for the rare ovulation class, using transition-aware sampling around phase boundaries, and producing calibrated prediction intervals instead of only point estimates for next-cycle onset. In a real application, richer daily-level inputs such as sleep, stress, symptoms, basal body temperature, or adherence patterns would also be important for more reliable and explainable predictions.

9. Conclusion

This project developed a leakage-safe multi-task LSTM pipeline for menstrual-cycle forecasting. The raw cycle-level data was expanded into daily records, converted into 21-day sliding-window sequences, and split at the user level to prevent user-specific leakage. The final shared-encoder model jointly predicted menstrual phase and days until the next cycle, achieving reasonable overall phase classification performance and a next-cycle MAE of approximately 5.14 days. The main limitation was poor ovulation detection, which is expected given the one-day minority-class label and approximate phase construction. Overall, the results show that the model can learn broad cycle progression, while future work should focus on rare phase transitions, uncertainty-aware prediction, and richer daily-level health signals.